

Universidad Autónoma de Tamaulipas

Facultad de Ingeniería Tampico



ASIGNATURA

Programación de Sistemas Base 2

9vo. Semestre

2026 -1

PROYECTO INTEGRADOR

Docente: Muñoz Quintero Dante Adolfo

Alumnos:

Jesus Antonio Duran Lugo
Saldaña Sánchez Carlos Alfonso
Sandoval Vázquez Cristhian Michel
Segura Alonso Adrián

Tabla de contenido

1. Descripción General.....	3
2. Objetivos	3
Objetivo General	3
Objetivos Específicos	4
3. Especificación Técnica	4
3.1 Extracción de Gramática	4
3.2 Estructura de Datos (Tokens y Nodos del AST)	5
3.3 Fases del Compilador	6
3.4 Manejo de Errores	7
4. Entregables del Repositorio	8
4.1 Estructura GitHub Actualizada.....	8
4.2 Contenidos por Carpeta	8
5. Demostración Funcional – Pruebas Requeridas.....	9

ESPECIFICACIÓN DEL PROYECTO: INTERPRETE ARITMÉTICO INTERACTIVO

1. Descripción General

Los estudiantes de la Facultad de Ingeniería Tampico presentan la documentación técnica evolutiva del **Intérprete o Compilador Aritmético Interactivo** desarrollado en Python. Este sistema está diseñado para recibir operaciones matemáticas escritas por el usuario en formato estándar (infijo), validar su estructura, calcular el resultado exacto, construir un Árbol de Sintaxis Abstracta (AST) y traducir de manera ortográfica todo el proceso al idioma español respetando las normas de la Real Academia Española (RAE).

El pipeline del compilador se compone formalmente de cuatro fases integradas:

- **Fase 1: Analizador Léxico (Lexer):** Fragmenta la cadena de texto de entrada en componentes léxicos legales (Tokens).
- **Fase 2: Analizador Sintáctico (Parser):** Valida la estructura gramatical mediante un esquema de descenso recursivo y construye la jerarquía del AST.
- **Fase 3: Análisis Semántico Integrado:** Controla microscópicamente la precisión por coma flotante, valida el límite estricto de rangos numéricos y restringe las operaciones de división entre cero.
- **Fase 4: Interpretación y Traducción Lingüística:** Ejecuta el recorrido del AST para calcular el valor real y genera de manera síncrona la traducción de la operación y el resultado final a palabras en español.

2. Objetivos

Objetivo General

Consolidar el desarrollo del intérprete aritmético interactivo mediante la formalización de su analizador sintáctico por descenso recursivo, el control semántico de excepciones y un backend de traducción lingüística automatizado, unificando todas las fases en un único entorno de ejecución en consola.

Objetivos Específicos

1. **Parser (Análisis Sintáctico):** Garantizar el procesamiento correcto de expresiones con operadores binarios, unarios y paréntesis, construyendo una estructura jerárquica enlazada que refleje fielmente la prioridad matemática.
2. **Análisis Semántico:** Interceptar en tiempo de compilación y ejecución desbordamientos aritméticos fuera del rango permitido (\$-999,999\$ a \$999,999\$) y operaciones matemáticamente indeterminadas (división por cero).
3. **Generación de Texto / Interpretación:** Diseñar un algoritmo recursivo de conversión numérica a texto que maneje de forma precisa las apócope del español ("un mil", "veintiún mil") y el deletreo aislado de componentes decimales.
4. **Interfaz e Inspección (Testing):** Proveer un menú interactivo con doble modo de operación para permitir tanto la traducción directa como la visualización en caracteres (ASCII) del árbol sintáctico generado.

3. Especificación Técnica

3.1 Extracción de Gramática

La sintaxis del Intérprete Aritmético Interactivo se rige por una gramática libre de contexto (CFG) diseñada para asegurar la precedencia de los operadores aritméticos tradicionales (\$*, /\$ sobre \$+, -\$) y la alteración de la geometría del árbol mediante paréntesis.

Se define la gramática formal mediante las siguientes producciones EBNF:

EBNF

expresion → **termino (("SUMA" | "RESTA") termino)***

termino → **factor (("MULT" | "DIV") factor)***

factor → **"LPAREN" expresion "RPAREN"**

| "RESTA" factor

| "SUMA" factor

| "NUM"

- **Precedencia Baja:** expresion procesa las sumas y restas de forma consecutiva (asociatividad por la izquierda).
- **Precedencia Media:** termino evalúa multiplicaciones y divisiones, llamando previamente a los factores atómicos.
- **Precedencia Alta (Átomo):** factor resuelve de manera recursiva la inclusión de expresiones entre paréntesis, operadores unarios negativos/positivos y la lectura directa de literales numéricas (NUM).

3.2 Estructura de Datos (Tokens y Nodos del AST)

Representación de Tokens

El componente léxico rompe la cadena plana de texto utilizando un enfoque basado en tuplas estructuradas (Tipo_de_Token, Valor_Texto) , gobernadas por las siguientes expresiones regulares:

Nombre del Token	Expresión Regular (Regex)	Descripción Técnica
NUM	<code>\d+\.\d+ \d+</code>	Números decimales o enteros (Prioriza coincidencia decimal).
SUMA	<code>\+</code>	Operador binario de adición.
RESTA	<code>-</code>	Operador binario de sustracción o unario negativo.
MULT	<code>*</code>	Operador binario de multiplicación.
DIV	<code>/</code>	Operador binario de división.
LPAREN	<code>\(</code>	Paréntesis de apertura izquierdo.
RPAREN	<code>\)</code>	Paréntesis de cierre derecho.
SKIP	<code>[\t]+</code>	Espacios y tabulaciones (Ignorados de la corriente final).
EOF	<i>Token Artificial</i>	Indica formalmente el fin de la cadena al Parser.

Jerarquía de Nodos del AST

El Árbol de Sintaxis Abstracta se modela mediante el paradigma orientado a objetos, donde cada clase abstracta implementa los métodos polimórficos `.evaluar()` y `.mostrar_arbol()`:

- **NodoNumero (Hoja Final):** Guarda el valor numérico en punto flotante (valor) y su equivalente ortográfico (texto). Su evaluación no procesa cálculos, solo retorna sus atributos base.
- **NodoOpBinaria (Rama Bifurcada):** Alberga las referencias a los subárboles izquierdo (izq) y derecho (der), además del operador binario (op). Al evaluarse, ejecuta recursividad profunda sobre sus dos hijos y concatena los textos con los conectores adecuados en español ("más", "menos", "multiplicado por", "dividido por").
- **NodoOpUnaria (Rama Monódica):** Modifica el signo de un único nodo interno. Multiplica el valor por -1 si el operador unario es 'RESTA' y antepone la palabra "menos" a la cadena de texto.

3.3 Fases del Compilador

El ciclo de procesamiento del flujo interactivo de traducción matemática se resume en la siguiente matriz arquitectónica:

Fase	Componente de Entrada	Componente de Salida	Responsabilidad Principal
Lexer (Analizador Léxico)	Cadena de caracteres introducida por la consola (string).	Lista indexada de Tuplas estructuradas (Tokens).	Filtrar espacios en blanco y agrupar los caracteres en lexemas legales. Detener el flujo ante símbolos ilegales (SyntaxError).
Parser (Analizador Sintáctico)	Lista de tokens generada por el lexer.	Objeto raíz enlazado del Árbol Sintáctico (arbol_ast).	Aplicar el descenso recursivo para validar la gramática aritmética y jerarquizar los nodos respetando los paréntesis y la precedencia.
Análisis Semántico (Integrado)	Valores calculados de forma	Excepciones controladas o flujo validado.	Interceptar la indeterminación por división entre cero (ZeroDivisionError) y restringir

	intermedia en los Nodos.		el procesamiento fuera del rango $[-999999, 999999]$ (ValueError). Redondear a 4 decimales para corregir imprecisiones de coma flotante.
Intérprete / Generador de Texto	Recorrido polimórfico del método <code>.evaluar()</code> sobre el AST.	Cadenas unificadas impresas en la terminal (Traducción Directa).	Traducir recursivamente las partes enteras y mapear de forma aislada (dígito por dígito) las partes decimales utilizando la palabra puente "punto".

3.4 Manejo de Errores

El sistema unifica la captura de excepciones en el bloque de arranque interactivo principal (try-except), garantizando que cualquier error detenga la compilación de la expresión actual de forma segura sin romper el bucle infinito del menú de usuario:

1. **Errores Léxicos y Sintácticos (SyntaxError):** * *Origen:* Caracteres extraños o no permitidos detectados por el Lexer , u omisión de delimitadores requeridos por el Parser (como paréntesis no balanceados).
 - *Ejemplo de salida:* [Error de Compilación]: Carácter ilegal detectado en la posición 0: a.
2. **Errores de Ejecución y Semánticos Matemáticos (ZeroDivisionError):**
 - *Origen:* El Parser permite la sintaxis de una división, pero el nodo derecho de la operación aritmética se evalúa dando un valor idéntico a cero.
 - *Ejemplo de salida:* [Error de Compilación]: No se puede dividir por cero..
3. **Errores de Validación de Límites (ValueError):**
 - *Origen:* El filtro semántico procesa un número real o decimal cuya magnitud absoluta excede el rango lingüístico seguro del diccionario ortográfico (un millón positivo o negativo).

- *Ejemplo de salida:* [Error de Compilación]: El número supera el rango permitido (-999,999 a 999,999)..

4. Entregables del Repositorio

4.1 Estructura GitHub Actualizada

interprete-numeros-letras/

	— main.py	# Punto de entrada principal y menú interactivo
	— lexer.py	# Analizador léxico y tokenización
	— parser_eval.py	# Analizador sintáctico y lógica de evaluación
	— nodos.py	# Definición de la jerarquía de nodos del AST
	— traductor.py	# Módulo semántico de traducción de números a

4.2 Contenidos por Carpeta

- Código fuente del intérprete modularizado con responsabilidades separadas.
- **main.py**: Orquestador que inicializa el bucle de ejecución, maneja las opciones de la consola (traducción directa e inspección de AST) y centraliza la captura de excepciones (SyntaxError, ZeroDivisionError, ValueError).
- **lexer.py**: Encargado de filtrar la cadena de entrada y generar la lista de tokens válidos.
- **parser_eval.py**: Consume los tokens para construir la estructura gramatical y coordinar el recorrido de resolución aritmética.
- **nodos.py**: Contiene la estructura de datos base para las hojas y ramas del Árbol Sintáctico.
- **traductor.py**: Aísla los algoritmos recursivos de conversión ortográfica y reglas de la RAE.

5. Demostración Funcional – Pruebas Requeridas

Traducción Directa:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python + v [icon] [icon] ... [icon] [icon] X

PS C:\Users\Jesus Antonio\Programas\Programacion de Sistemas Base1\Proyecto1> & "C:\Users\Jesus Antonio\AppData\Local\Programs\Python\Python313\python.exe" "c:/Users/Jesus Antonio/Programas/Programacion de Sistemas Base1/Proyecto1/compilador4.py"

COMPILADOR INTERACTIVO CON PARÉNTESIS Y DECIMAL
1. Modo Traducción Directa
2. Modo Inspección de Árbol Sintáctico (AST)
3. Salir

Selecciona una opción (1-3): 1

Introduce la operación (ej: (1 + 2) * 3): 52*9*625/8

--- RESULTADO DE LA TRADUCCIÓN ---
-> Resultado Real: 36562.5
-> Operación Evaluada: cincuenta y dos multiplicado por nueve multiplicado por seiscientos veinticinco dividido por ocho
-> Resultado en Letras: treinta y seis mil quinientos sesenta y dos punto cinco

COMPILADOR INTERACTIVO CON PARÉNTESIS Y DECIMAL
1. Modo Traducción Directa
2. Modo Inspección de Árbol Sintáctico (AST)
3. Salir

Selecciona una opción (1-3): |

Ln 123, Col 27 Spaces: 4 UTF-8 CRLF Python Python 3.13 (64-bit) Go Live [icon] [icon]
```

Error. Rango invalido de números:

```
Selecciona una opción (1-3): 1

Introduce la operación (ej: (1 + 2) * 3): 4158796+54896

[Error de Compilación]: El número supera el rango permitido (-999,999 a 999,999).

COMPILADOR INTERACTIVO CON PARÉNTESIS Y DECIMAL
1. Modo Traducción Directa
2. Modo Inspección de Árbol Sintáctico (AST)
3. Salir

Selecciona una opción (1-3): |

Ln 123, Col 27
```

Error. Carácter no valido:

```
Selecciona una opción (1-3): asdhfkagf75847

Opción no válida. Intenta de nuevo.

COMPILADOR INTERACTIVO CON PARÉNTESIS Y DECIMAL
1. Modo Traducción Directa
2. Modo Inspección de Árbol Sintáctico (AST)
3. Salir

Selecciona una opción (1-3): █
```

Error. Instrucción de menú no valida:

```
PS C:\Users\Jesus Antonio\Programas\Programacion de Sistemas Base1\InterpreteDeNumeros_a_Letras> & "C:\Users\Jesus Antonio\AppData\Local\Programs\Python\Python313\python.exe" "c:/Users/Jesus Antonio/Programas/Programacion de Sistemas Base1/InterpreteDeNumeros_a_Letras/main.py"

COMPILADOR INTERACTIVO CON PARÉNTESIS Y DECIMAL
1. Modo Traducción Directa
2. Modo Inspección de Árbol Sintáctico (AST)
3. Salir

Selecciona una opción (1-3): 4

Opción no válida. Intenta de nuevo.

COMPILADOR INTERACTIVO CON PARÉNTESIS Y DECIMAL
1. Modo Traducción Directa
2. Modo Inspección de Árbol Sintáctico (AST)
3. Salir

Selecciona una opción (1-3): █
```

Modo inspección del árbol sintáctico

```
Introduce la operación (ej: (1 + 2) * 3): 458/967*(85/96+47)

--- ÁRBOL DE SINTAXIS ABSTRACTA (AST) ---
├── OP_BINARIA (MULT)
│   ├── OP_BINARIA (DIV)
│   │   ├── NUM (458.0) -> cuatrocientos cincuenta y ocho
│   │   └── NUM (967.0) -> novecientos sesenta y siete
│   └── OP_BINARIA (SUMA)
│       ├── OP_BINARIA (DIV)
│       │   ├── NUM (85.0) -> ochenta y cinco
│       │   └── NUM (96.0) -> noventa y seis
│       └── NUM (47.0) -> cuarenta y siete
-> Evaluación matemática del árbol: 22.679959496725267
```

Información del curso

- **Materia:** Programación de Sistemas Base 2
- **Institución:** Universidad Autónoma de Tamaulipas
- **Semestre:** 2026-2
- **Profesor:** Muñoz Quintero Dante Adolfo

Integrantes

NOMBRE	MATRÍCULA
JESUS ANTONIO DURAN LUGO	2213332156
SALDAÑA SÁNCHEZ CARLOS ALFONSO	22033330162
SANDOVAL VÁZQUEZ CRISTHIAN MICHEL	2193390093
SEGURA ALONSO ADRIÁN	2213332212

Descripción del Lenguaje

Intérprete y traductor interactivo desarrollado en Python. Diseñado para calcular expresiones matemáticas exactas con jerarquía de operaciones y traducir el resultado y el proceso a palabras en español.

Ejemplo:

Introduce la operación (ej: $(1 + 2) * 3$): $(1.5 + 2) * -3$

-> Resultado Real: -10.5

-> Operación Evaluada: uno punto cinco más dos multiplicado por menos tres

-> Resultado en Letras: menos diez punto cinco

Fases del Compilador

1. **Lexer:** Tokenización de la cadena de entrada.
2. **Parser :** Análisis sintáctico y construcción del Árbol de Sintaxis Abstracta.
3. **Traductor (Semántico):** Validación de rangos (\$-999,999\$ a \$999,999\$) y traducción ortográfica.
4. **Intérprete/Interfaz:** Menú en consola para ejecución y visualización gráfica del AST.

Cómo Ejecutar

Requisitos

- Python 3.13 (o compatible).

Compilar y Ejecutar

- Primero Se dirige al repositorio de github, después descarga la carpeta completa y en cualquier herramienta para correr código abren esa carpeta y la ejecutan en el archivo main.py

Estructura del Proyecto

InterpreteDeNumeros_a_Letras

```
├── leer.py          # Analizador léxico y definición de tokens
├── main.py          # Menú interactivo y manejo de errores centralizado
├── nodos.py         # Clases: NodoNumero, NodoOpBinaria, NodoOpUnaria
├── numero_a_letra.py # Algoritmo de traducción semántica y ortográfica
└── parser.py        # Analizador sintáctico por descenso recursivo
```

Tokens Reconocidos

- **Operadores:** +, -, *, /.
- **Delimitadores:** (,).
- **Literales:** Números (enteros y decimales).
- **Ignorados:** Espacios y tabulaciones (SKIP).

Reportes de Error

El compilador reporta errores en tiempo real deteniendo la ejecución sin cerrar el programa:

[Error de Compilación]: Carácter ilegal detectado en la posición 0: a

[Error de Compilación]: El número supera el rango permitido (-999,999 a 999,999).

[Error de Compilación]: No se puede dividir por cero.